

Midterm Individual Exercise 5(Binary Tree Orientation Exercise)

Objective:

Create a program that illustrates a use-case of the Tree data structure.

Create a program for a Reference class for a node of a typical Binary Tree.

Create a program for a Typical Binary Tree.

Required:

1. Create a Java project that has a package that contains the classes TreeNode, Tree and TreeTest as shown below.

Name of Project: <CourseNumber><CourseCode><YourName>SampleTree

Example: CS211_9300_DelacruzJuan_SampleTree

2. Test the main class

REQUIRED:

Show the output through a multiline comment in the source code. The multiline comment shows what appears on the output screen when main class is executed. You should also put appropriate comments in appropriate places in the source code.

Deliverable:

Upload zipped project folder in the Submission Bin for this requirement

Name of Folder/Project: <CourseNumber><CourseCode><YourName>SampleTree.zip
CS211_9300_DelacruzJuan_SampleTree

Given Programs:

```
/**
The following class defines a template for a node of a binary search tree.*/
public class TreeNode<T> {
    private TreeNode<T> leftNode;
    private T data;
    private TreeNode<T> rightNode;

    public TreeNode(T nodeData){
        data = nodeData;
        leftNode = null;
        rightNode = null;
    }
    public void setLeft(TreeNode<T> left) {
        leftNode = left;
    }
    public void setRight(TreeNode<T> right) {
        rightNode = right;
    }
    public void setData(T d) {
        data = d;
    }
    public TreeNode<T> getLeftNode() {
        return leftNode;
    }
    public TreeNode<T> getRightNode(){
        return rightNode;
    }
    public T getData(){
        return data;
    }
    public String toString() {
        return data.toString();
    }
    public void insert(T insertValue) {
        if (insertValue.toString().compareTo(data.toString())<0) {
            if ( leftNode == null )
                leftNode = new TreeNode<T>(insertValue);
            else
                leftNode.insert(insertValue);
        } // end if
        else
            if (insertValue.toString().compareTo(data.toString())>0) {
                if (rightNode == null)
                    rightNode = new TreeNode<T>(insertValue);
                else
                    rightNode.insert(insertValue);
            } // end else if
        } // end insert
    } // end TreeNode<T> class
}
```

/**

Sample Binary Search Tree Class

*/

```
public class Tree<T> {
    private TreeNode<T> root;

    public Tree(){
        root = null;
    }

    public void insertNode(T insertValue) {
        if (root == null)
            root = new TreeNode<T>(insertValue);
        else
            root.insert(insertValue);
    }

    public void preorderTraversal() {
        preorderHelper(root);
    }

    private void preorderHelper(TreeNode<T> node) {
        if (node == null)
            return;
        System.out.print(node.getData().toString()+" ");
        preorderHelper(node.getLeftNode());
        preorderHelper(node.getRightNode());
    }

    public void inorderTraversal() {
        inorderHelper(root);
    }

    private void inorderHelper(TreeNode<T> node ) {
        if (node == null)
            return;
        inorderHelper(node.getLeftNode());
        System.out.print(node.getData().toString()+ " ");
        inorderHelper(node.getRightNode());
    }

    public void postorderTraversal() {
        postorderHelper(root);
    }

    private void postorderHelper( TreeNode<T> node ) {
        if ( node == null )
            return;
        postorderHelper(node.getLeftNode());
        postorderHelper(node.getRightNode());
        System.out.print(node.getData().toString()+" ");
    }
} // end class Tree
```

```

/**
 * Test class using the Tree class defined above
 * The following is a main class that uses the Tree class, thereby, the
 * TreeNode class
 */

import java.io.*;

public class TreeTest {
    public void run() throws Exception {
        Tree<Integer> tree = new Tree<Integer>();
        Integer value;

        System.out.println("Inserting the following values: ");

        for (int i = 1; i <= 10; i++) {
            value = (new Integer((int) (Math.random()*100)));
            System.out.print(value.toString()+" ");
            tree.insertNode(value);
        }

        System.out.println("\n\nPreorder traversal");
        tree.preorderTraversal();

        System.out.println("\n\nInorder traversal");
        tree.inorderTraversal();

        System.out.println("\n\nPostorder traversal");
        tree.postorderTraversal();
        System.out.println();
    }

    public static void main(String[] args) {
        TreeTest myTest;
        try {
            myTest = new TreeTest();
            myTest.run();
        } catch (Exception e)
        {
            e.printStackTrace();
        }
        System.exit(0);
    } // end main
} // end of TreeTest class

```